Guided Lab 03: Deployments – From Basics to Rolling Updates

Welcome to Deployments

Previously, we saw how ReplicaSets help keep Pods alive. But in a real-world application, that's just the beginning.

In production, we need:

- Version control
- Easy updates with zero downtime
- Rollbacks if things go wrong
- Seamless scaling

All of this comes from a **Deployment**, not just a ReplicaSet.

> So in this lab, we're going to start simple — then layer in powerful features step by step.

Step 1: Set Up Your Working Directory

```
cd ~/k8s-labs
mkdir -p deployments
cd deployments
```

Part 1: A Simple Deployment (Your Production Starting Point)

Let's start with a basic NGINX Deployment with 2 replicas.

Step 1: Create the file

```
touch nginx-deployment.yaml
nano nginx-deployment.yaml
```

Step 2: Add the following content

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21.1
          ports:
            - containerPort: 80
```

Step 3: Apply it

```
kubectl apply -f nginx-deployment.yaml
```

Then verify:

```
kubectl get deployments
kubectl get replicasets
kubectl get pods
```

You'll notice:

- Kubernetes **created a ReplicaSet automatically**
- That ReplicaSet created **2 Pods**
- This is the **standard production pattern**

So while you don't *see* a ReplicaSet in your YAML, it's **happening under the hood**.

Part 2: Improve It for Rolling Updates (how changes are rolled)

Deployment Update Strategies:

a. RollingUpdate (default)

- Pods are updated gradually with zero downtime.
- Uses maxUnavailable and maxSurge to control rollout speed.

Step 1: improve your file

```
nano nginx-deployment.yaml
```

With: strategy

```
  strategy:
    type: RollingUpdate
    rollingUpdate:
```

```
      maxSurge: 1
      maxUnavailable: 0
```

Update the spec to include:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21.1
          ports:
            - containerPort: 80
```

This tells Kubernetes:

- **maxSurge: 1** → allow 1 extra Pod during updates
- **maxUnavailable: 0** → ensure no downtime

Reapply it:

```
kubectl apply -f nginx-deployment.yaml
```

b. Recreate

- Deletes all old Pods first, then creates new ones.

- Causes downtime during the transition.

```
strategy:
  type: Recreate
```

Use when: App can't run multiple versions at once (e.g., shared DB).

Part 3: Demo a Rolling Update

Let's simulate deploying a new version of our app.

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.23.1
```

Watch the rollout:

```
kubectl rollout status deployment/nginx-deployment
```

What happens:

- A new ReplicaSet is created
- Pods are updated **one by one**
- No downtime occurs due to the rolling strategy

Check the status:

```
kubectl get pods
kubectl get replicasets
```

Part 4: Roll Back the Update

What if this version was buggy?

Just undo it:

```
kubectl rollout undo deployment/nginx-deployment
```

Verify rollback:

```
kubectl get pods
kubectl rollout history deployment/nginx-deployment
```

Kubernetes brings you back to the previous stable version. This is **version control for your infrastructure.**

Part 5: Scale with a Single Command

Want 5 replicas?

```
kubectl scale deployment nginx-deployment --replicas=5
```

Verify:

```
kubectl get pods
```

Scale down to 2 again:

```
kubectl scale deployment nginx-deployment --replicas=2
```

Wrap-Up: What Did You Learn?

- **Deployments** are the standard for running production apps
- They manage ReplicaSets behind the scenes
- They support:
  - **Rolling updates** to upgrade apps with no downtime
  - **Rollbacks** if something goes wrong
  - **Scaling** with a single command

In one YAML file, you're getting a resilient, self-healing, version-controlled deployment system — that’s why we use Deployments in production, not Pods or ReplicaSets alone.